CS-UY 3923/CS-GY 6813: Information Security & Privacy

Evan Johnson, Fall 2026, Lecture 3

Memory Corruption II: Heap Corruption

Logistics

- Assignment 1 was due today (should be graded soon)
- Assignment 2 is out, due next week (any issues so far?)
  - Gradescope link is up
  - If you have any trouble accessing Gradescope, message Will or Aryan
- Logistics questions?

If your name is on this list, please sign the attendance form.

- Isabel Ayres, Ege Tepegoz, Zirui Xu, Munirah Althefeery, Anissa Braca, Ketan Joya, Habib Khoja, Varshith Reddy Lingala, Aaditya Rengarajan
- If you think you signed the form in the past, I probably couldn't read your handwriting

Extra credit opportunity!

- Take a bike lock.
- Bring it back open, get some extra credit points.
- Rules:
  - One lock per group
  - Sign the form so I know who has what
  - 2-week limit on opening the lock
  - No permanent damage!

Last Week

- Buffer overflows, command injection, and integer overflows
- Practicalities of writing shellcode

Today: Exploiting advanced memory vulnerabilities

Memory management in C/C++

- C and C++ store local variables on the stack
- But what about data that needs to stick around longer than a single function?

Memory management in C/C++

C

```c
int *p = malloc(4 * sizeof(int));
p[0] = 42;
printf("%d\n", p[0]);
free(p);
```

C++

```cpp
int *p = new int[4];
p[0] = 42;
std::cout << p[0] << "\n";
delete[] p;
```

Memory management in C/C++

C

```c
int *p = malloc(4 * sizeof(int));
p[0] = 42;
printf("%d\n", p[0]);
free(p);
```

C++

```cpp
int *p = new int[4];
p[0] = 42;
std::cout << p[0] << "\n";
delete[] p;
```

Poll: Who has run into problems with memory allocation before? What happened?

How can memory management go wrong?

- We could forget to free it (memory leak)

```c
int *p = malloc(4 * sizeof(int));
p[0] = 42;
printf("%d\n", p[0]);
```

How can memory management go wrong?

- We could forget to free it (memory leak)
- We could accidentally free it twice (double free)

```c
int *p = malloc(4 * sizeof(int));
p[0] = 42;
printf("%d\n", p[0]);
free(p);
free(p);
```

How can memory management go wrong?

- We could forget to free it (memory leak)
- We could accidentally free it twice (double free)
- We could use it after we free it (use-after-free)

```c
int *p = malloc(4 * sizeof(int));
p[0] = 42;
free(p);
printf("%d\n", p[0]);
```

How can memory management go wrong?

- We could forget to free it (memory leak)
- We could accidentally free it twice (double free)
- We could use it after we free it (use-after-free)
- We could (of course) access it out-of-bounds

```c
int *p = malloc(4 * sizeof(int));
p[4] = 42;
printf("%d\n", p[4]);
free(p);
```

Memory management in C is hard

- The programmer is responsible for the details
- The system does not track memory liveness (no garbage collection)
- The system does not ensure that pointers are valid

Memory management in a real program

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define NBUCKETS 256
#define MAX_LINE 4096

struct entry {
    char *key;
    char *val;
    struct entry *next;
};

struct cache {
    struct entry **buckets;
};

struct request {
    char *verb;
    char **args;
    int nargs;
};

struct session {
    int fd;
    char *inbuf;
    size_t inlen;
    struct request *pending;
};

static int running = 1;

static unsigned hash(const char *s) {
    unsigned h = 5381;
    while (*s) h = h * 33 + (unsigned char)*s++;
    return h % NBUCKETS;
}

static char *copy_str(const char *s, size_t n) {
    char *out = malloc(n + 1);
    if (!out) return NULL;
    memcpy(out, s, n);
    out[n] = '\0';
    return out;
}

struct cache *cache_new(void) {
    struct cache *c = malloc(sizeof(*c));
    if (!c) return NULL;
    c->buckets = calloc(NBUCKETS, sizeof(struct entry *));
    if (!c->buckets) {
        free(c);
        return NULL;
    }
    return c;
}
```

```c
static void entry_free(struct entry *e) {
    free(e->key);
    free(e->val);
    free(e);
}

struct entry *cache_get(struct cache *c, const char *key) {
    struct entry *e = c->buckets[hash(key)];
    for (; e; e = e->next) {
        if (strcmp(e->key, key) == 0) return e;
    }
    return NULL;
}

int cache_set(struct cache *c, const char *key, const char *val) {
    struct entry *e = cache_get(c, key);
    if (e) {
        char *nv = copy_str(val, strlen(val));
        if (!nv) return -1;
        free(e->val);
        e->val = nv;
        return 0;
    }
    e = malloc(sizeof(*e));
    if (!e) return -1;
    e->key = copy_str(key, strlen(key));
    e->val = copy_str(val, strlen(val));
    if (!e->key || !e->val) {
        free(e->key);
        free(e->val);
        free(e);
        return -1;
    }
    e->next = c->buckets[hash(key)];
    c->buckets[hash(key)] = e;
    return 0;
}

void cache_destroy(struct cache *c) {
    for (int i = 0; i < NBUCKETS; i++) {
        while (c->buckets[i]) {
            struct entry *e = c->buckets[i];
            c->buckets[i] = e->next;
            entry_free(e);
        }
    }
    free(c->buckets);
    free(c);
}

static void request_free(struct request *r) {
    if (!r) return;
    for (int i = 0; i < r->nargs; i++) free(r->args[i]);
    free(r->args);
    free(r->verb);
    free(r);
}
```

```c
}

static struct request *request_parse(const char *line) {
    struct request *r = calloc(1, sizeof(*r));
    if (!r) return NULL;
    const char *p = line;
    while (*p) {
        while (*p == ' ') p++;
        const char *start = p;
        while (*p && *p != ' ') p++;
        if (p == start) break;
        char *tok = copy_str(start, (size_t)(p - start));
        if (!tok) goto fail;
        if (!r->verb) {
            r->verb = tok;
            continue;
        }
        char **grown = realloc(r->args, (r->nargs + 1) * sizeof(char *));
        if (!grown) {
            free(tok);
            goto fail;
        }
        r->args = grown;
        r->args[r->nargs++] = tok;
    }
    return r;
fail:
    request_free(r);
    return NULL;
}

struct session *session_open(int fd) {
    struct session *s = calloc(1, sizeof(*s));
    if (!s) return NULL;
    s->fd = fd;
    s->inbuf = malloc(MAX_LINE);
    if (!s->inbuf) {
        free(s);
        return NULL;
    }
    return s;
}

void session_close(struct session *s) {
    running = 0;
    request_free(s->pending);
    free(s->inbuf);
    close(s->fd);
    free(s);
}

static int session_fill(struct session *s) {
    if (s->inlen == MAX_LINE) return -1;
    ssize_t n = read(s->fd, s->inbuf + s->inlen, MAX_LINE - s->inlen);
    if (n <= 0) return -1;
    s->inlen += (size_t)n;
    return 0;
```

```c
}

static void reply(struct session *s, const char *status, const char *body) {
    size_t n = strlen(status) + strlen(body) + 3;
    char *msg = malloc(n);
    if (!msg) return;
    snprintf(msg, n, "%s %s\n", status, body);
    write(s->fd, msg, n - 1);
    free(msg);
}

static void dispatch(struct session *s, struct cache *c) {
    struct request *r = s->pending;
    if (strcmp(r->verb, "GET") == 0 && r->nargs == 1) {
        struct entry *e = cache_get(c, r->args[0]);
        reply(s, e ? "OK" : "ERR", e ? e->val : "not found");
    } else if (strcmp(r->verb, "SET") == 0 && r->nargs == 2) {
        int rc = cache_set(c, r->args[0], r->args[1]);
        reply(s, rc == 0 ? "OK" : "ERR", r->args[0]);
    } else if (strcmp(r->verb, "QUIT") == 0) {
        session_close(s);
        return;
    } else {
        reply(s, "ERR", "bad command");
    }
    request_free(s->pending);
    s->pending = NULL;
}

void session_pump(struct session *s, struct cache *c) {
    if (session_fill(s) < 0) {
        session_close(s);
        return;
    }
    char *nl;
    while ((nl = memchr(s->inbuf, '\n', s->inlen)) != NULL) {
        size_t linelen = (size_t)(nl - s->inbuf);
        char *line = copy_str(s->inbuf, linelen);
        if (!line) break;
        memmove(s->inbuf, nl + 1, s->inlen - linelen - 1);
        s->inlen -= linelen + 1;
        s->pending = request_parse(line);
        free(line);
        if (s->pending && s->pending->verb) dispatch(s, c);
    }
}

int main(void) {
    struct cache *c = cache_new();
    struct session *s = session_open(STDIN_FILENO);
    if (!c || !s) return 1;
    while (running) session_pump(s, c);
    cache_destroy(c);
    return 0;
}
```

Memory management in a real program

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define NBUCKETS 256
#define MAX_LINE 4096

struct entry {
    char *key;
    char *val;
    struct entry *next;
};

struct cache {
    struct entry **buckets;
};

struct request {
    char *verb;
    char **args;
    int nargs;
};

struct session {
    int fd;
    char *inbuf;
    size_t inlen;
    struct request *pending;
};

static int running = 1;

static unsigned hash(const char *s) {
    unsigned h = 5381;
    while (*s) h = h * 33 + (unsigned char)*s++;
    return h % NBUCKETS;
}

static char *copy_str(const char *s, size_t n) {
    char *out = malloc(n + 1);
    if (!out) return NULL;
    memcpy(out, s, n);
    out[n] = '\0';
    return out;
}

struct cache *cache_new(void) {
    struct cache *c = malloc(sizeof(*c));
    if (!c) return NULL;
    c->buckets = calloc(NBUCKETS, sizeof(struct entry *));
    if (!c->buckets) {
        free(c);
        return NULL;
    }
    return c;
}
```

```c
static void entry_free(struct entry *e) {
    free(e->key);
    free(e->val);
    free(e);
}

struct entry *cache_get(struct cache *c, const char *key) {
    struct entry *e = c->buckets[hash(key)];
    for (; e; e = e->next) {
        if (strcmp(e->key, key) == 0) return e;
    }
    return NULL;
}

int cache_set(struct cache *c, const char *key, const char *val) {
    struct entry *e = cache_get(c, key);
    if (e) {
        char *nv = copy_str(val, strlen(val));
        if (!nv) return -1;
        free(e->val);
        e->val = nv;
        return 0;
    }
    e = malloc(sizeof(*e));
    if (!e) return -1;
    e->key = copy_str(key, strlen(key));
    e->val = copy_str(val, strlen(val));
    if (!e->key || !e->val) {
        free(e->key);
        free(e->val);
        free(e);
        return -1;
    }
    e->next = c->buckets[hash(key)];
    c->buckets[hash(key)] = e;
    return 0;
}

void cache_destroy(struct cache *c) {
    for (int i = 0; i < NBUCKETS; i++) {
        while (c->buckets[i]) {
            struct entry *e = c->buckets[i];
            c->buckets[i] = e->next;
            entry_free(e);
        }
    }
    free(c->buckets);
    free(c);
}

static void request_free(struct request *r) {
    if (!r) return;
    for (int i = 0; i < r->nargs; i++) free(r->args[i]);
    free(r->args);
    free(r->verb);
    free(r);
}
```

```c
}

static struct request *request_parse(const char *line) {
    struct request *r = calloc(1, sizeof(*r));
    if (!r) return NULL;
    const char *p = line;
    while (*p) {
        while (*p == ' ') p++;
        const char *start = p;
        while (*p && *p != ' ') p++;
        if (p == start) break;
        char *tok = copy_str(start, (size_t)(p - start));
        if (!tok) goto fail;
        if (!r->verb) {
            r->verb = tok;
            continue;
        }
        char **grown = realloc(r->args, (r->nargs + 1) * sizeof(char *));
        if (!grown) {
            free(tok);
            goto fail;
        }
        r->args = grown;
        r->args[r->nargs++] = tok;
    }
    return r;
fail:
    request_free(r);
    return NULL;
}

struct session *session_open(int fd) {
    struct session *s = calloc(1, sizeof(*s));
    if (!s) return NULL;
    s->fd = fd;
    s->inbuf = malloc(MAX_LINE);
    if (!s->inbuf) {
        free(s);
        return NULL;
    }
    return s;
}

void session_close(struct session *s) {
    running = 0;
    request_free(s->pending);
    free(s->inbuf);
    close(s->fd);
    free(s);
}

static int session_fill(struct session *s) {
    if (s->inlen == MAX_LINE) return -1;
    ssize_t n = read(s->fd, s->inbuf + s->inlen, MAX_LINE - s->inlen);
    if (n <= 0) return -1;
    s->inlen += (size_t)n;
    return 0;
```

```c
}

static void reply(struct session *s, const char *status, const char *body) {
    size_t n = strlen(status) + strlen(body) + 3;
    char *msg = malloc(n);
    if (!msg) return;
    snprintf(msg, n, "%s %s\n", status, body);
    write(s->fd, msg, n - 1);
    free(msg);
}

static void dispatch(struct session *s, struct cache *c) {
    struct request *r = s->pending;
    if (strcmp(r->verb, "GET") == 0 && r->nargs == 1) {
        struct entry *e = cache_get(c, r->args[0]);
        reply(s, e ? "OK" : "ERR", e ? e->val : "not found");
    } else if (strcmp(r->verb, "SET") == 0 && r->nargs == 2) {
        int rc = cache_set(c, r->args[0], r->args[1]);
        reply(s, rc == 0 ? "OK" : "ERR", r->args[0]);
    } else if (strcmp(r->verb, "QUIT") == 0) {
        session_close(s);
        return;
    } else {
        reply(s, "ERR", "bad command");
    }
    request_free(s->pending);
    s->pending = NULL;
}

void session_pump(struct session *s, struct cache *c) {
    if (session_fill(s) < 0) {
        session_close(s);
        return;
    }
    char *nl;
    while ((nl = memchr(s->inbuf, '\n', s->inlen)) != NULL) {
        size_t linelen = (size_t)(nl - s->inbuf);
        char *line = copy_str(s->inbuf, linelen);
        if (!line) break;
        memmove(s->inbuf, nl + 1, s->inlen - linelen - 1);
        s->inlen -= linelen + 1;
        s->pending = request_parse(line);
        free(line);
        if (s->pending && s->pending->verb) dispatch(s, c);
    }
}

int main(void) {
    struct cache *c = cache_new();
    struct session *s = session_open(STDIN_FILENO);
    if (!c || !s) return 1;
    while (running) session_pump(s, c);
    cache_destroy(c);
    return 0;
}
```

Most modern memory vulns are heap-based

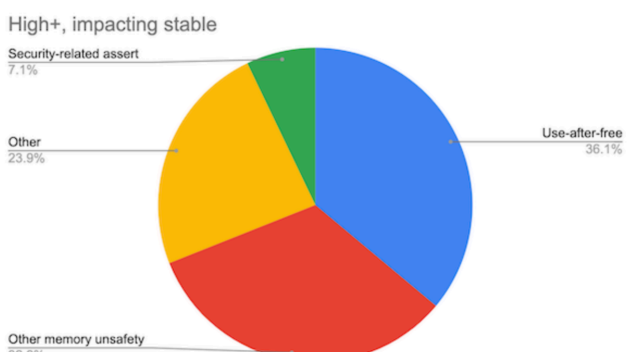
Chromium > Chromium Security >

Memory safety

The Chromium project finds that around 70% of our serious security bugs are memory safety problems. Our next major project is to prevent such bugs at source.

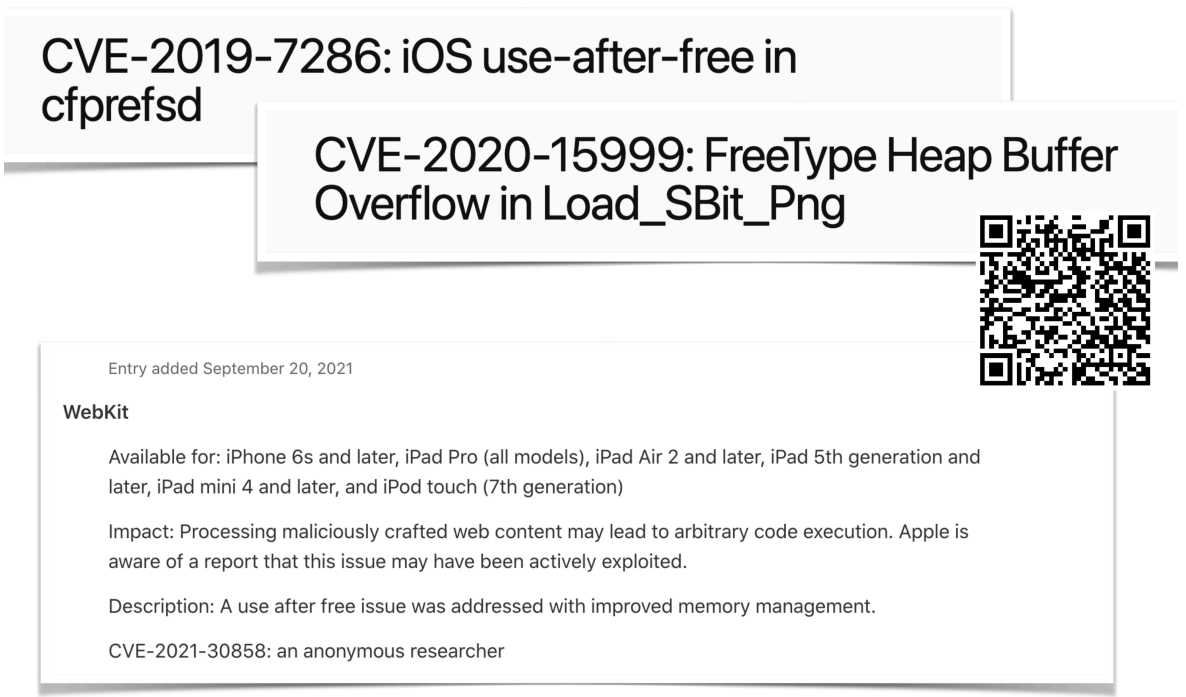
The problem

Around 70% of our high severity security bugs are memory unsafety problems (that is, mistakes with C/C++ pointers). Half of *those* are use-after-free bugs.



(Analysis based on 912 high or critical severity security bugs since 2015, affecting the Stable channel.)

Most modern memory vulns are heap-based



The Plan

- Memory allocators: How do they actually work?
- Heap corruption: How are allocation bugs exploited?
- Case Study: Apache HTTP/2 Exploit

The Plan

- Memory allocators: How do they actually work?
- Heap corruption: How are allocation bugs exploited?
- Case Study: Apache HTTP/2 Exploit

Dynamic Memory: Abstraction

- **Dynamically allocate and release memory buffers as needed. Magic!**
  - `ptr=malloc(20)`: **Give me 20 bytes.**
  - `free(ptr)`: **I'm done with my 20 bytes. Take them back.**

Dynamic Memory: Reality

- Where does this memory come from?
- How does “the system” know how much memory to reclaim when free() is called?
- Details vary by heap implementation, but follow a common pattern

What do you know about how dynamic memory works in practice?

Dynamic memory comes from the heap

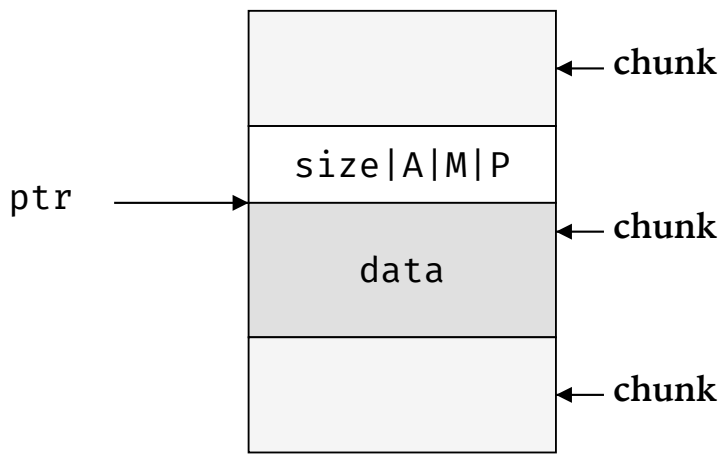
- The heap is just a big block of memory in the process address space.
- Managed by the memory allocator
- There are many different allocators, even on a single system
  - Some are optimized for allocations of a certain size, for speed, for space efficiency, etc.

Heap basics

- The heap is organized into “chunks”, the basic unit of memory
- If a chunk is in use, it corresponds to 1 allocation (a `malloc`)
- If a chunk is free, it corresponds to a contiguous block of memory to be used for future allocations

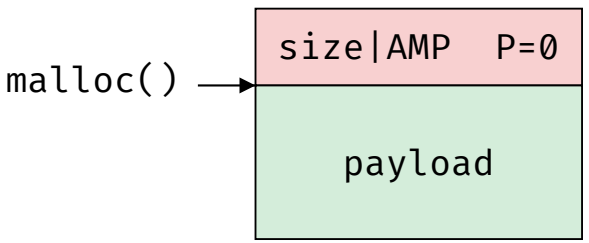
What's a chunk?

- Metadata: size (8-byte aligned; 16 on 64-bit) + flags
  - size
  - A: Non-main arena (not relevant for us)
  - M: Is mmaped (not relevant for us)
  - P: Previous chunk is in-use
- Has slightly different structure based on whether it is in-use or free



In-use chunk

- `malloc` returns pointer to the payload
- How does `free` know how much to free?
  - Look at the metadata: chunk size

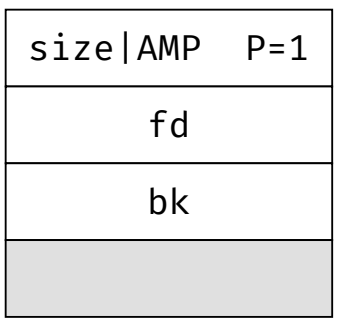


Attaching metadata to these chunks seems very convenient.

What could go wrong with this design?

Free chunk

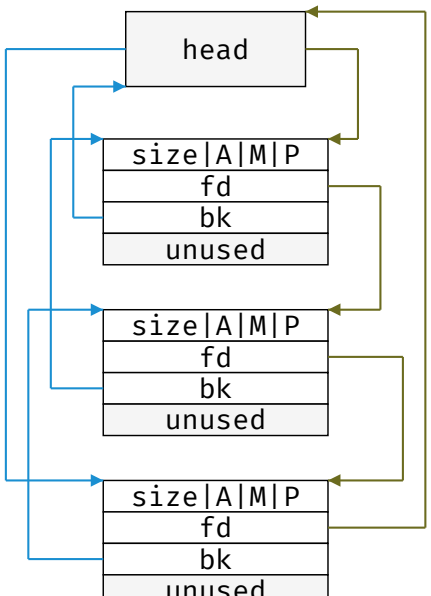
- Free chunks are kept in a doubly linked list
  - Unused payload data is used to store link pointers
- Consecutive free chunks are combined (no two free chunks can be adjacent to each other)



Why do free chunks need to track `fd` and `bk` but in-use chunks don't?

Free chunks are kept in a circular “Free List”

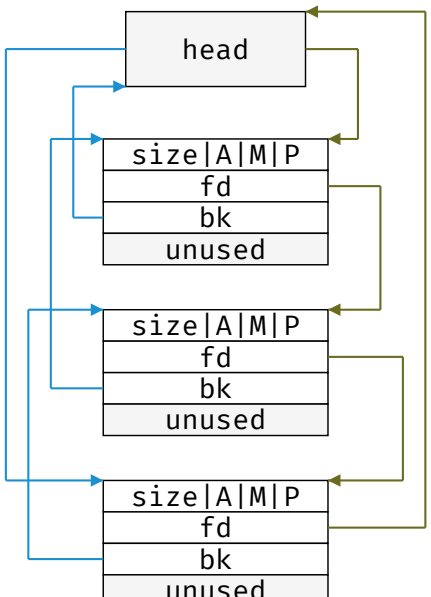
- Chunks are inserted into the list when they are freed.
- Chunks are removed from the list if they get allocated, or if they need to be combined with a newly-freed adjacent chunk



Free chunks are kept in a circular “Free List”

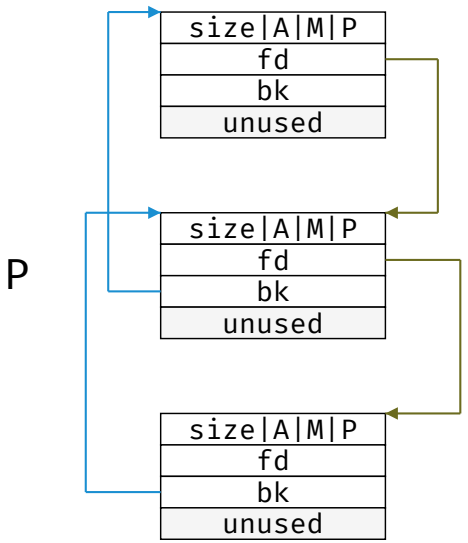
- Chunks are inserted into the list when they are freed.
- Chunks are removed from the list if they get allocated, or if they need to be combined with a newly-freed adjacent chunk
- Unlink operation to remove a chunk from the free list:

```c
#define unlink(P, BK, FD) {
    FD = P->fd;
    BK = P->bk;
    FD->bk = BK;
    BK->fd = FD;
}
```



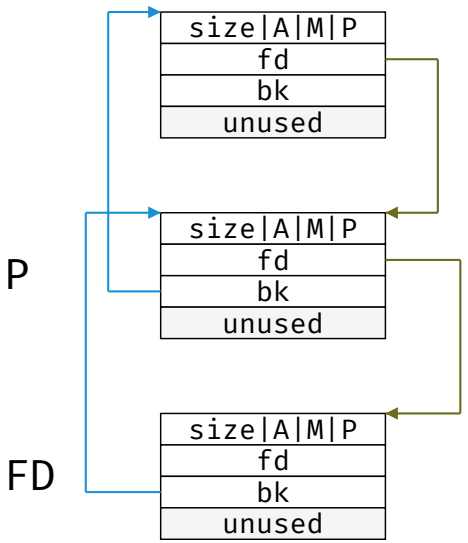
What happens when we update the free list?

```c
#define unlink(P, BK, FD) {
    FD = P->fd;
    BK = P->bk;
    FD->bk = BK;
    BK->fd = FD;
}
```



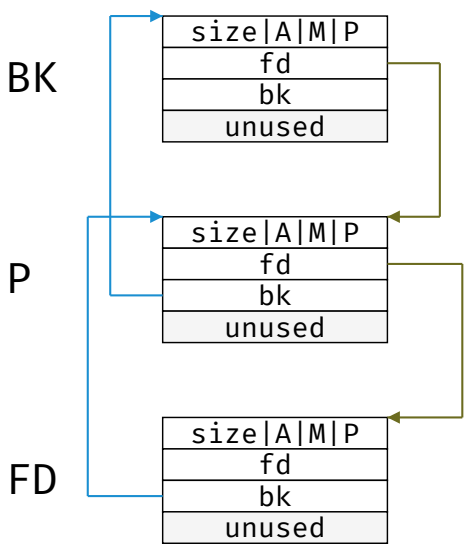
What happens when we update the free list?

```c
#define unlink(P, BK, FD) {
    FD = P->fd;
    BK = P->bk;
    FD->bk = BK;
    BK->fd = FD;
}
```



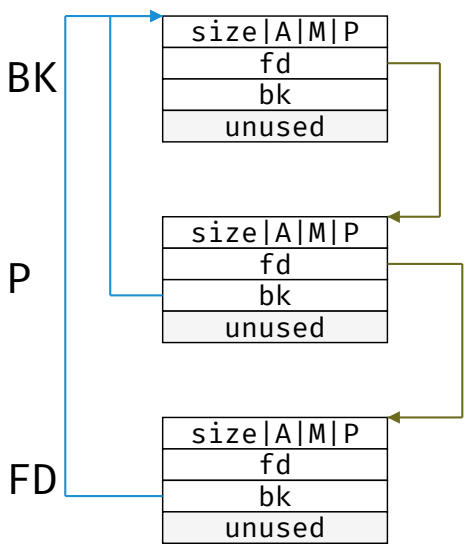
What happens when we update the free list?

```c
#define unlink(P, BK, FD) {
    FD = P->fd;
    BK = P->bk;
    FD->bk = BK;
    BK->fd = FD;
}
```



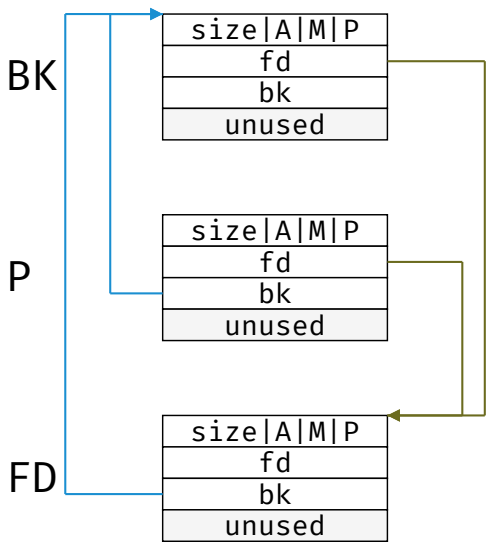
What happens when we update the free list?

```c
#define unlink(P, BK, FD) {
    FD = P->fd;
    BK = P->bk;
    FD->bk = BK;
    BK->fd = FD;
}
```



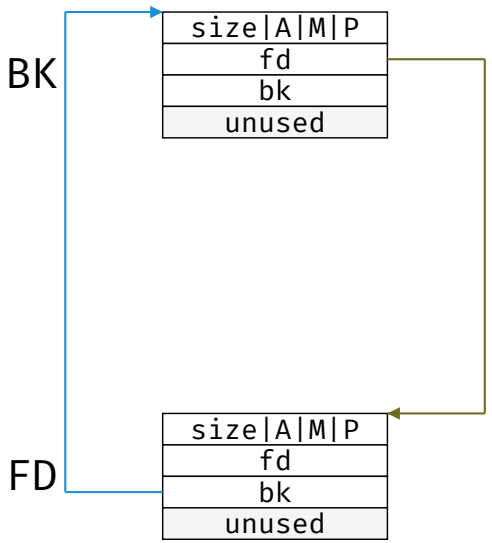
What happens when we update the free list?

```c
#define unlink(P, BK, FD) {
    FD = P->fd;
    BK = P->bk;
    FD->bk = BK;
    BK->fd = FD;
}
```



What happens when we update the free list?

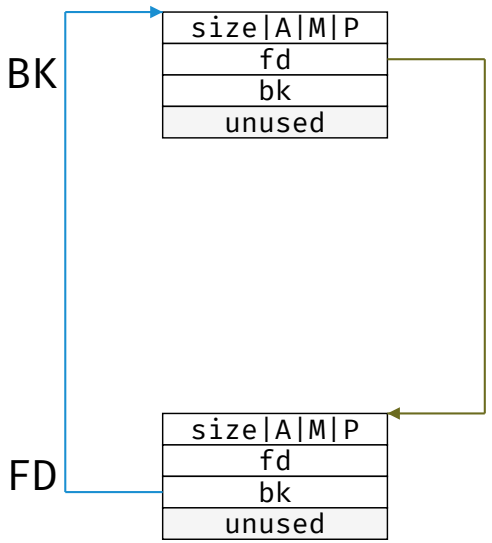
```c
#define unlink(P, BK, FD) {
    FD = P->fd;
    BK = P->bk;
    FD->bk = BK;
    BK->fd = FD;
}
```



What happens when we update the free list?

```c
#define unlink(P, BK, FD) {
    FD = P->fd;
    BK = P->bk;
    FD->bk = BK;
    BK->fd = FD;
}
```

What could go wrong with the unlink operation?



Review: Memory allocators

- The heap is a big block of memory managed by the allocator
- The heap maintains a data structure—the free list—to track memory
- The free list gets updated every time we allocate/free memory

The Plan

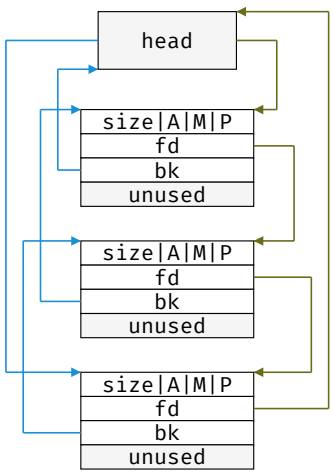
- Memory allocators: How do they actually work?
- **Heap corruption: How are allocation bugs exploited?**
- Case Study: Apache HTTP/2 Exploit

Heap Corruption

- Unlike corrupting the stack, we can't just overwrite a return pointer
- Instead, we try more subtle techniques:
  - What can we do if we manage to get `free()` to act on data we control?
  - What if we can cause the heap manager to act on fake chunks?

The key to heap exploits: manipulating metadata

- The heap has its own data structures to track free and allocated memory
- If you overwrite one of those data structures, the heap's memory management code will still use them as though they were valid
- This may be possible, since attacker data is often stored on the heap



But how can an attacker control metadata?

- Heap buffer overflows
- Double free
- Use after free

Heap attack 1: Heap overflow

- Similar mechanism to stack overflow: write beyond bounds of data structure
- More complicated to exploit in practice: on the stack, you can directly overwrite the return pointer, but on the heap, there is an extra level of indirection through the heap metadata

Spot the vulnerability

```C
static void do_syslog(int pri, char *msg) {
  char *p, *tmp, save;

  for (p = msg; /* ... iterate over line chunks ... */) {
      if (strlen(p) > MAXSYSLOGLEN) {
          // Find a word boundary to break the line
          for (tmp = p + MAXSYSLOGLEN; tmp > p && *tmp ≠ ' '; tmp--);
          if (tmp ⩽ p)
              tmp = p + MAXSYSLOGLEN;

          save = *tmp;
          *tmp = '\0';
          SYSLOG(pri, "%s", p);
          *tmp = save;

          for (p = tmp; *p ≠ ' '; p++);
          *p = '\0';
      } else {
          SYSLOG(pri, "%s", p);
          break;
      }
```

Spot the vulnerability

```C
static void do_syslog(int pri, char *msg) {
  char *p, *tmp, save;

  for (p = msg; /* ... iterate over line chunks ... */) {
      if (strlen(p) > MAXSYSLOGLEN) {
          // Find a word boundary to break the line
          for (tmp = p + MAXSYSLOGLEN; tmp > p && *tmp ≠ ' '; tmp--);
          if (tmp ⩽ p)
              tmp = p + MAXSYSLOGLEN;

          save = *tmp;
          *tmp = '\0';
          SYSLOG(pri, "%s", p);
          *tmp = save;

          for (p = tmp; *p ≠ ' '; p++);
          *p = '\0';
      } else {
          SYSLOG(pri, "%s", p);
          break;
      }
```

Spot the vulnerability

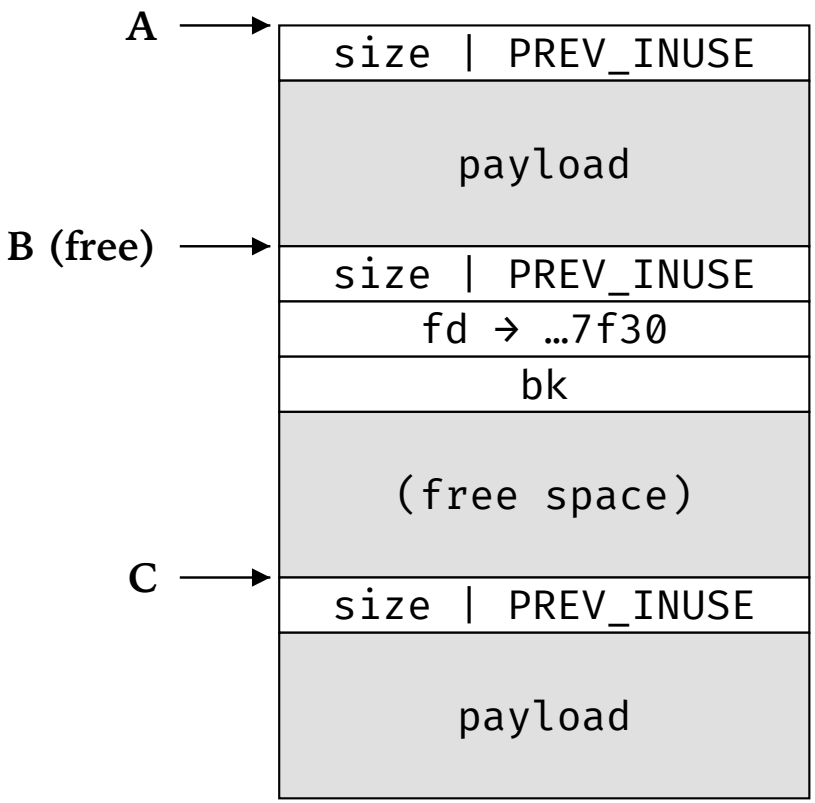
```C
static void do_syslog(int pri, char *msg) {
  char *p, *tmp, save;

  for (p = msg; /* ... iterate over line chunks ... */) {
      if (strlen(p) > MAXSYSLOGLEN) {
          // Find a word boundary to break the line
          for (tmp = p + MAXSYSLOGLEN; tmp > p && *tmp ≠ ' '; tmp--);
          if (tmp ⩽ p)
              tmp = p + MAXSYSLOGLEN;

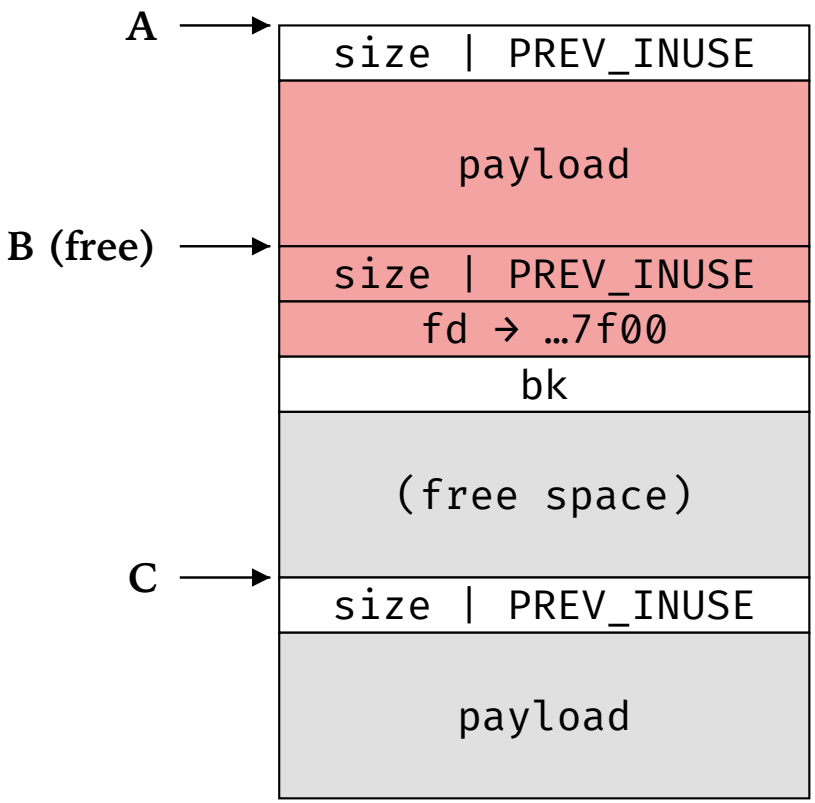
          save = *tmp;
          *tmp = '\0';
          SYSLOG(pri, "%s", p);
          *tmp = save;

          for (p = tmp; *p ≠ ' '; p++);
          *p = '\0';
      } else {
          SYSLOG(pri, "%s", p);
          break;
      }
```

One NUL byte turns unlink into a write primitive

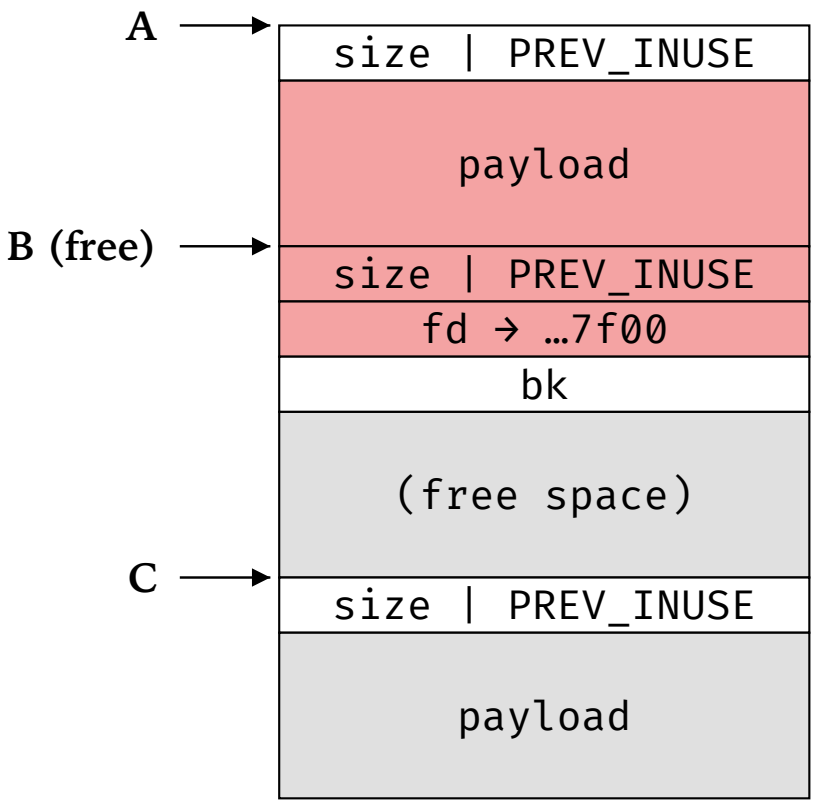


One NUL byte turns unlink into a write primitive



1-byte NUL overflow: the write off the end of A clears the low byte of `fd` (`…7f30` ⟶ `…7f00`).

One NUL byte turns unlink into a write primitive




```c
#define unlink(P, BK, FD) {
    FD = P→fd;
    BK = P→bk;
    FD→bk = BK;
    BK→fd = FD;
}
```

Heap overflows can be quite complicated!

- Unlike stack overflows, they often take multiple steps
- You sometimes need the heap to be in a certain configuration for them to work

Heap attack 2: Double free

- For each `malloc`, we need to `free` exactly once.
- What happens when we don't?

```c
int *p = malloc(4 * sizeof(int));
p[0] = 42;
printf("%d\n", p[0]);
free(p);
free(p);
```

Double frees can corrupt heap metadata

```C
a = malloc(10);
b = malloc(10);
c = malloc(10);
free(a);
free(b);
free(a);
d = malloc(10); // uses memory returned from free(a)
e = malloc(10); // uses memory returned from free(b)
f = malloc(10); // uses memory returned from free(a)... now d points to f
```

Spot the vulnerability

```c
void cleanup_request(Request *req) {
    if (req->body) {
        free(req->body);
    }
}

void cleanup_buffer(Buffer *buf) {
    if (buf->data) {
        free(buf->data);
    }
}

Buffer *make_buffer_from_request(Request *req) {
    Buffer *buf = malloc(sizeof(Buffer));
    buf->data = req->body;
    return buf;
}
```

Exploiting double frees

- Now you have aliased chunks in the free list
- Can be application dependent how you can actually pivot to code execution
  - Example: With two overlapping chunks, you can convince program to overwrite function pointers on the heap
- Most consistent exploit method is via vtable corruption

Review: Polymorphism (virtual methods)

Python

```python
class Cow:
    def speak(self):
        print("Moo!")

class Dog:
    def speak(self):
        print("Woof!")

def make_noise(animal):
    animal.speak()

cow = Cow()
dog = Dog()

make_noise(cow)  # Moo!
make_noise(dog)  # Woof!
```

C++

```cpp
class Animal {
public:
    virtual void speak() = 0;
};

class Cow : public Animal {
    void speak() override { std::cout << "Moo!\n"; }
};

class Dog : public Animal {
    void speak() override { std::cout << "Woof!\n"; }
};

void make_noise(const Animal& animal) {
    animal.speak();
}

int main() {
    Cow cow;
    Dog dog;
    make_noise(cow);  // Moo!
    make_noise(dog);  // Woof!
}
```

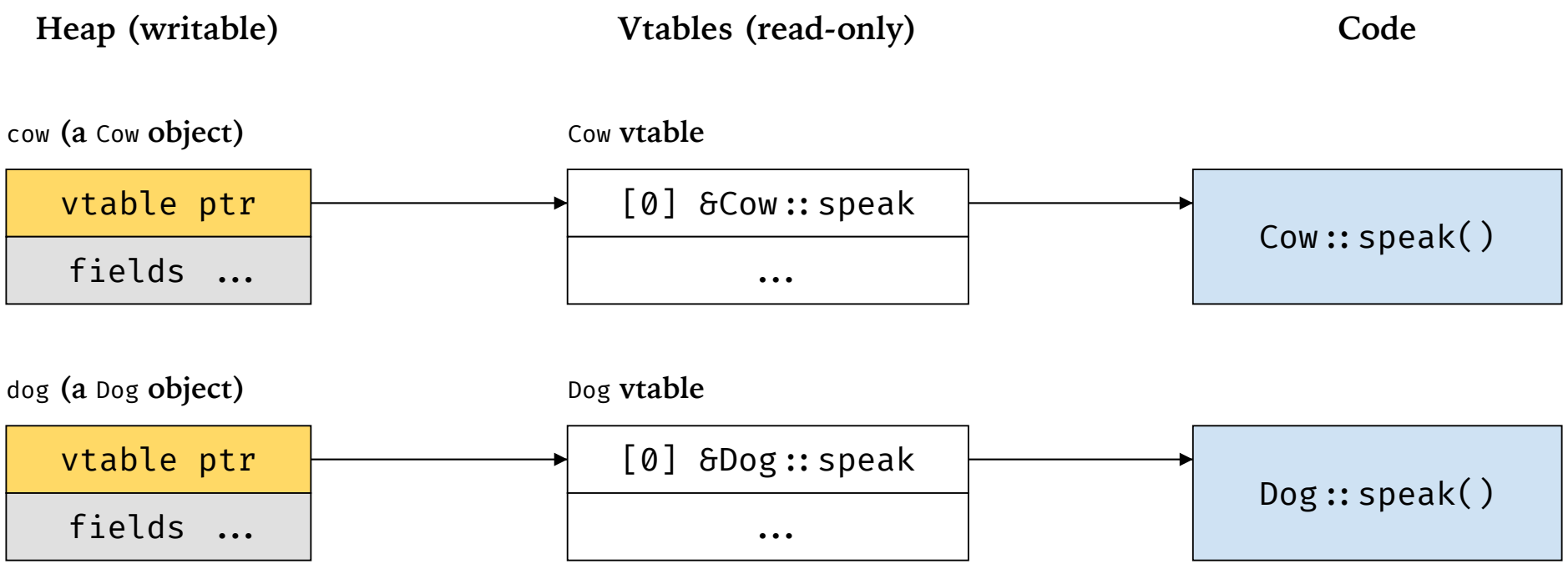
How do C++ classes call virtual methods?

- What's the abstraction?
- How is it implemented in reality?
  - When `animal.speak()` is called from within `make_noise()`, how is control transferred to the correct implementation of `speak()`?

Vtables

- Each object contains a pointer to its vtable
- Vtable is an array of function pointers (1 entry per function)
- `make_noise()` compiles to something like: `(*animal→vtable[0])(animal)`
- Based on the class and function, the compiler knows which offset in which vtable to use

Vtables in memory



`animal.speak()` → **follow the object's vtable ptr, call entry** `[0]`

How do vtables relate to heap exploits?

Vtables → heap is full of function pointers

- Relatively straightforward attack vector for C++ code
- Example: Double free causes heap chunk to be aliased
  - First chunk holds `Dog` object
  - Second chunk overwrites first chunk with 0x41414141…
  - Code calls `dog.speak()` → data at 0x41414141 gets called
  - Control flow hijacked!

Heap attack 3: Use-after-free

- Most common (modern) memory safety vulnerability
- Extremely common in web browsers
- When code uses memory that has already been freed, it allows allocation aliasing
  - Can exploit just like a double free

What will be printed here?

```c
char *p;
p = malloc(20);
snprintf(p, 20, "Hi");
printf("%s\n", p);
free(p);
printf("%s\n", p);
```

What will be printed here?

```c
char *p;
p = malloc(20);
snprintf(p, 20, "Hi");
printf("%s\n", p);
free(p);
printf("%s\n", p);
```

- Note that in a multi-threaded application, the second malloc() may not be obvious.
- Just because the use of previously freed memory appears right after the free does not mean it is safe.

The key to finding UAFs

- UAFs generally occur when an object is freed, but other pointers to it remain
- Two very common patterns based on this idea:
  - Several containers have pointers to data, but only 1 is freed
  - Event handler has access to object, but it is freed elsewhere

Spot the vulnerability (1)

```c
typedef struct Node {
    char key[16];
    int value;
    struct Node *next;
} Node;

Node *table_head = NULL;      // linked list
Node *last_used = NULL;       // 1-entry cache of the last lookup

// Insert a new node at the head of the list
void table_insert(const char *key, int value) {
    Node *n = malloc(sizeof(Node));
    strncpy(n->key, key, sizeof(n->key));
    n->value = value;
    n->next = table_head;
    table_head = n;
}
```

```c
// Walk the list looking for `key`; populate the cache on a hit
Node *table_lookup(const char *key) {
    if (last_used && strcmp(last_used->key, key) == 0)
        return last_used;               // fast path: cache hit

    for (Node *n = table_head; n; n = n->next) {
        if (strcmp(n->key, key) == 0) {
            last_used = n;              // remember it for next time
            return n;
        }
    }
    return NULL;
}

// Remove `key` from the list and free its node
void table_delete(const char *key) {
    Node **cur = &table_head;
    while (*cur) {
        if (strcmp((*cur)->key, key) == 0) {
            Node *victim = *cur;
            *cur = victim->next;        // unlink from the list
            free(victim);
            return;
        }
        cur = &(*cur)->next;
    }
}
```

- What does this code do?

Spot the vulnerability (1)

```c
typedef struct Node {
    char key[16];
    int value;
    struct Node *next;
} Node;

Node *table_head = NULL;       // linked list
Node *last_used = NULL;        // 1-entry cache of the last lookup

// Insert a new node at the head of the list
void table_insert(const char *key, int value) {
    Node *n = malloc(sizeof(Node));
    strncpy(n->key, key, sizeof(n->key));
    n->value = value;
    n->next = table_head;
    table_head = n;
}
```

```c
// Walk the list looking for `key`; populate the cache on a hit
Node *table_lookup(const char *key) {
    if (last_used && strcmp(last_used->key, key) == 0)
        return last_used;                // fast path: cache hit

    for (Node *n = table_head; n; n = n->next) {
        if (strcmp(n->key, key) == 0) {
            last_used = n;               // remember it for next time
            return n;
        }
    }
    return NULL;
}

// Remove `key` from the list and free its node
void table_delete(const char *key) {
    Node **cur = &table_head;
    while (*cur) {
        if (strcmp((*cur)->key, key) == 0) {
            Node *victim = *cur;
            *cur = victim->next;         // unlink from the list
            free(victim);
            return;
        }
        cur = &(*cur)->next;
    }
}
```

Spot the vulnerability (1)

```c
typedef struct Node {
    char key[16];
    int value;
    struct Node *next;
} Node;

Node *table_head = NULL;      // linked list
Node *last_used = NULL;       // 1-entry cache of the last lookup

// Insert a new node at the head of the list
void table_insert(const char *key, int value) {
    Node *n = malloc(sizeof(Node));
    strncpy(n->key, key, sizeof(n->key));
    n->value = value;
    n->next = table_head;
    table_head = n;
}
```

```c
// Walk the list looking for `key`; populate the cache on a hit
Node *table_lookup(const char *key) {
    if (last_used && strcmp(last_used->key, key) == 0)
        return last_used;                // fast path: cache hit

    for (Node *n = table_head; n; n = n->next) {
        if (strcmp(n->key, key) == 0) {
            last_used = n;               // remember it for next time
            return n;
        }
    }
    return NULL;
}

// Remove `key` from the list and free its node
void table_delete(const char *key) {
    Node **cur = &table_head;
    while (*cur) {
        if (strcmp((*cur)->key, key) == 0) {
            Node *victim = *cur;
            *cur = victim->next;         // unlink from the list
            free(victim);
            return;
        }
        cur = &(*cur)->next;
    }
}
```

Spot the vulnerability (1)

```c
typedef struct Node {
    char key[16];
    int value;
    struct Node *next;
} Node;

Node *table_head = NULL;      // linked list
Node *last_used = NULL;       // 1-entry cache of the last lookup

// Insert a new node at the head of the list
void table_insert(const char *key, int value) {
    Node *n = malloc(sizeof(Node));
    strncpy(n->key, key, sizeof(n->key));
    n->value = value;
    n->next = table_head;
    table_head = n;
}
```

```c
// Walk the list looking for `key`; populate the cache on a hit
Node *table_lookup(const char *key) {
    if (last_used && strcmp(last_used->key, key) == 0)
        return last_used;                // fast path: cache hit

    for (Node *n = table_head; n; n = n->next) {
        if (strcmp(n->key, key) == 0) {
            last_used = n;               // remember it for next time
            return n;
        }
    }
    return NULL;
}

// Remove `key` from the list and free its node
void table_delete(const char *key) {
    Node **cur = &table_head;
    while (*cur) {
        if (strcmp((*cur)->key, key) == 0) {
            Node *victim = *cur;
            *cur = victim->next;         // unlink from the list
            free(victim);
            return;
        }
        cur = &(*cur)->next;
    }
}
```

Spot the vulnerability (1)

```c
typedef struct Node {
    char key[16];
    int value;
    struct Node *next;
} Node;

Node *table_head = NULL;      // linked list
Node *last_used = NULL;       // 1-entry cache of the last lookup

// Insert a new node at the head of the list
void table_insert(const char *key, int value) {
    Node *n = malloc(sizeof(Node));
    strncpy(n->key, key, sizeof(n->key));
    n->value = value;
    n->next = table_head;
    table_head = n;
}
```

```c
// Walk the list looking for `key`; populate the cache on a hit
Node *table_lookup(const char *key) {
    if (last_used && strcmp(last_used->key, key) == 0)
        return last_used;                // fast path: cache hit

    for (Node *n = table_head; n; n = n->next) {
        if (strcmp(n->key, key) == 0) {
            last_used = n;               // remember it for next time
            return n;
        }
    }
    return NULL;
}

// Remove `key` from the list and free its node
void table_delete(const char *key) {
    Node **cur = &table_head;
    while (*cur) {
        if (strcmp((*cur)->key, key) == 0) {
            Node *victim = *cur;
            *cur = victim->next;         // unlink from the list
            free(victim);
            return;
        }
        cur = &(*cur)->next;
    }
}
```

Spot the vulnerability (1)

```c
typedef struct Node {
    char key[16];
    int value;
    struct Node *next;
} Node;

Node *table_head = NULL;      // linked list
Node *last_used = NULL;       // 1-entry cache of the last lookup

// Insert a new node at the head of the list
void table_insert(const char *key, int value) {
    Node *n = malloc(sizeof(Node));
    strncpy(n->key, key, sizeof(n->key));
    n->value = value;
    n->next = table_head;
    table_head = n;
}
```

```c
// Walk the list looking for `key`; populate the cache on a hit
Node *table_lookup(const char *key) {
    if (last_used && strcmp(last_used->key, key) == 0)
        return last_used;               // fast path: cache hit

    for (Node *n = table_head; n; n = n->next) {
        if (strcmp(n->key, key) == 0) {
            last_used = n;              // remember it for next time
            return n;
        }
    }
    return NULL;
}

// Remove `key` from the list and free its node
void table_delete(const char *key) {
    Node **cur = &table_head;
    while (*cur) {
        if (strcmp((*cur)->key, key) == 0) {
            Node *victim = *cur;
            *cur = victim->next;        // unlink from the list
            free(victim);
            return;
        }
        cur = &(*cur)->next;
    }
}
```

- So where's the bug?

Spot the vulnerability (1)

```c
typedef struct Node {
    char key[16];
    int value;
    struct Node *next;
} Node;

Node *table_head = NULL;       // linked list
Node *last_used = NULL;        // 1-entry cache of the last lookup

// Insert a new node at the head of the list
void table_insert(const char *key, int value) {
    Node *n = malloc(sizeof(Node));
    strncpy(n->key, key, sizeof(n->key));
    n->value = value;
    n->next = table_head;
    table_head = n;
}
```

```c
// Walk the list looking for `key`; populate the cache on a hit
Node *table_lookup(const char *key) {
    if (last_used && strcmp(last_used->key, key) == 0)
        return last_used;                  // fast path: cache hit

    for (Node *n = table_head; n; n = n->next) {
        if (strcmp(n->key, key) == 0) {
            last_used = n;                 // remember it for next time
            return n;
        }
    }
    return NULL;
}

// Remove `key` from the list and free its node
void table_delete(const char *key) {
    Node **cur = &table_head;
    while (*cur) {
        if (strcmp((*cur)->key, key) == 0) {
            Node *victim = *cur;
            *cur = victim->next;           // unlink from the list
            free(victim);
            return;
        }
        cur = &(*cur)->next;
    }
}
```

Spot the vulnerability (2)

```c
typedef struct Request Request;

typedef struct {
    Request *req;
    void (*on_complete)(Request *req, int bytes_read);
} AsyncReadOp;

struct Request {
    int client_fd;
    char *buffer;
    int pending_io;  // # async ops still in flight for this request
};

AsyncReadOp *pending_ops[8];
int pending_count = 0;
```

```c
void queue_async_read(Request *req, void (*cb)(Request *, int)) {
    AsyncReadOp *op = malloc(sizeof(AsyncReadOp));
    op->req = req;
    op->on_complete = cb;
    pending_ops[pending_count++] = op;
    req->pending_io++;
}

// Runs later, when the event loop decides the read is ready
void run_pending_ops(void) {
    for (int i = 0; i < pending_count; i++) {
        AsyncReadOp *op = pending_ops[i];
        op->on_complete(op->req, 128);
        free(op);
    }
    pending_count = 0;
}

// Triggers when connection times out
void handle_client_error(Request *req) {
    fprintf(stderr, "client fd=%d timed out\n", req->client_fd);
    free(req->buffer);
    free(req);
}
```

- What does this code do?

Spot the vulnerability (2)

```c
typedef struct Request Request;

typedef struct {
    Request *req;
    void (*on_complete)(Request *req, int bytes_read);
} AsyncReadOp;

struct Request {
    int client_fd;
    char *buffer;
    int pending_io;  // # async ops still in flight for this request
};

AsyncReadOp *pending_ops[8];
int pending_count = 0;
```

```c
void queue_async_read(Request *req, void (*cb)(Request *, int)) {
    AsyncReadOp *op = malloc(sizeof(AsyncReadOp));
    op->req = req;
    op->on_complete = cb;
    pending_ops[pending_count++] = op;
    req->pending_io++;
}

// Runs later, when the event loop decides the read is ready
void run_pending_ops(void) {
    for (int i = 0; i < pending_count; i++) {
        AsyncReadOp *op = pending_ops[i];
        op->on_complete(op->req, 128);
        free(op);
    }
    pending_count = 0;
}

// Triggers when connection times out
void handle_client_error(Request *req) {
    fprintf(stderr, "client fd=%d timed out\n", req->client_fd);
    free(req->buffer);
    free(req);
}
```

Spot the vulnerability (2)

```c
typedef struct Request Request;

typedef struct {
    Request *req;
    void (*on_complete)(Request *req, int bytes_read);
} AsyncReadOp;

struct Request {
    int client_fd;
    char *buffer;
    int pending_io;  // # async ops still in flight for this request
};

AsyncReadOp *pending_ops[8];
int pending_count = 0;
```

```c
void queue_async_read(Request *req, void (*cb)(Request *, int)) {
    AsyncReadOp *op = malloc(sizeof(AsyncReadOp));
    op->req = req;
    op->on_complete = cb;
    pending_ops[pending_count++] = op;
    req->pending_io++;
}

// Runs later, when the event loop decides the read is ready
void run_pending_ops(void) {
    for (int i = 0; i < pending_count; i++) {
        AsyncReadOp *op = pending_ops[i];
        op->on_complete(op->req, 128);
        free(op);
    }
    pending_count = 0;
}

// Triggers when connection times out
void handle_client_error(Request *req) {
    fprintf(stderr, "client fd=%d timed out\n", req->client_fd);
    free(req->buffer);
    free(req);
}
```

Spot the vulnerability (2)

```c
typedef struct Request Request;

typedef struct {
    Request *req;
    void (*on_complete)(Request *req, int bytes_read);
} AsyncReadOp;

struct Request {
    int client_fd;
    char *buffer;
    int pending_io;  // # async ops still in flight for this request
};

AsyncReadOp *pending_ops[8];
int pending_count = 0;
```

```c
void queue_async_read(Request *req, void (*cb)(Request *, int)) {
    AsyncReadOp *op = malloc(sizeof(AsyncReadOp));
    op->req = req;
    op->on_complete = cb;
    pending_ops[pending_count++] = op;
    req->pending_io++;
}

// Runs later, when the event loop decides the read is ready
void run_pending_ops(void) {
    for (int i = 0; i < pending_count; i++) {
        AsyncReadOp *op = pending_ops[i];
        op->on_complete(op->req, 128);
        free(op);
    }
    pending_count = 0;
}

// Triggers when connection times out
void handle_client_error(Request *req) {
    fprintf(stderr, "client fd=%d timed out\n", req->client_fd);
    free(req->buffer);
    free(req);
}
```

Spot the vulnerability (2)

```c
typedef struct Request Request;

typedef struct {
    Request *req;
    void (*on_complete)(Request *req, int bytes_read);
} AsyncReadOp;

struct Request {
    int client_fd;
    char *buffer;
    int pending_io;  // # async ops still in flight for this request
};

AsyncReadOp *pending_ops[8];
int pending_count = 0;
```

```c
void queue_async_read(Request *req, void (*cb)(Request *, int)) {
    AsyncReadOp *op = malloc(sizeof(AsyncReadOp));
    op->req = req;
    op->on_complete = cb;
    pending_ops[pending_count++] = op;
    req->pending_io++;
}

// Runs later, when the event loop decides the read is ready
void run_pending_ops(void) {
    for (int i = 0; i < pending_count; i++) {
        AsyncReadOp *op = pending_ops[i];
        op->on_complete(op->req, 128);
        free(op);
    }
    pending_count = 0;
}

// Triggers when connection times out
void handle_client_error(Request *req) {
    fprintf(stderr, "client fd=%d timed out\n", req->client_fd);
    free(req->buffer);
    free(req);
}
```

- So where's the bug?

Spot the vulnerability (2)

```c
typedef struct Request Request;

typedef struct {
    Request *req;
    void (*on_complete)(Request *req, int bytes_read);
} AsyncReadOp;

struct Request {
    int client_fd;
    char *buffer;
    int pending_io;  // # async ops still in flight for this request
};

AsyncReadOp *pending_ops[8];
int pending_count = 0;
```

```c
void queue_async_read(Request *req, void (*cb)(Request *, int)) {
    AsyncReadOp *op = malloc(sizeof(AsyncReadOp));
    op->req = req;
    op->on_complete = cb;
    pending_ops[pending_count++] = op;
    req->pending_io++;
}

// Runs later, when the event loop decides the read is ready
void run_pending_ops(void) {
    for (int i = 0; i < pending_count; i++) {
        AsyncReadOp *op = pending_ops[i];
        op->on_complete(op->req, 128);
        free(op);
    }
    pending_count = 0;
}

// Triggers when connection times out
void handle_client_error(Request *req) {
    fprintf(stderr, "client fd=%d timed out\n", req->client_fd);
    free(req->buffer);
    free(req);
}
```

An aside: What if we just don’t allow reallocations?

Review: Heap Vulnerabilities

- Heap bugs can be subtle and hard to prevent.
- 3 basic sorts of heap vulnerabilities: overflows, double free, and use-after-free.
- Some good targets for heap vulns:
  - Variables used in security checks (e.g., buffer_size)
  - Function pointers
  - vTable pointers

The Plan

- Memory allocators: How do they actually work?
- Heap corruption: How are allocation bugs exploited?
- **Case Study: Apache HTTP/2 Exploit**

Case study: Apache HTTP/2 exploit

- Apache is an extremely popular open-source web server
- The bug we're looking at allows pre-auth RCE
  - i.e., any attacker can take over your machine
- Why it's interesting:
  - This bug is very modern (early 2026)
  - Apache is very security-hardened
  - This bug should convince you how scary heap vulns are

The setup

- Apache implements HTTP/2
- HTTP/2 allows multiple concurrent streams on 1 connection
- This bug occurs when we spawn a new stream (using a HEADERS frame) and immediately reset that stream (using a RST_STREAM frame)
- Because Apache has not had time to fully set up the new stream, it accidentally cleans it up twice

Two paths to the same cleanup function

Path 1: the `RST_STREAM` frame handler

```c
case NGHTTP2_RST_STREAM:
    /* ... */
    h2_mplx_c1_client_rst(session->mplx, frame->hd.stream_id, stream);
    break;
```

Path 2: nghttp2's `on_stream_close` callback (generic cleanup run whenever a stream closes)

```c
static int on_stream_close_cb(nghttp2_session *ngh2, int32_t stream_id,
                              uint32_t error_code, void *userp)
{
    h2_session *session = (h2_session *)userp;
    h2_stream *stream = get_stream(session, stream_id);
    if (stream) {
        if (error_code) {
            h2_stream_rst(stream, error_code);
            h2_mplx_c1_client_rst(session->mplx, stream_id, stream);
        }
    }
    return 0;
}
```

Two paths to the same cleanup function

Path 1: the `RST_STREAM` frame handler

```c
case NGHTTP2_RST_STREAM:
    /* ... */
    h2_mplx_c1_client_rst(session->mplx, frame->hd.stream_id, stream);
    break;
```

Path 2: nghttp2's `on_stream_close` callback (generic cleanup run whenever a stream closes)

```c
static int on_stream_close_cb(nghttp2_session *ngh2, int32_t stream_id,
                              uint32_t error_code, void *userp)
{
    h2_session *session = (h2_session *)userp;
    h2_stream *stream = get_stream(session, stream_id);
    if (stream) {
        if (error_code) {
            h2_stream_rst(stream, error_code);
            h2_mplx_c1_client_rst(session->mplx, stream_id, stream);
        }
    }
    return 0;
}
```

The reset cleanup function

```C
apr_status_t h2_mplx_c1_client_rst(h2_mplx *m, int stream_id, h2_stream *stream)
{
    apr_status_t status = APR_SUCCESS;
    int registered;

    H2_MPLX_ENTER_ALWAYS(m);
    registered = (h2_ihash_get(m->streams, stream_id) != NULL);
    if (!stream) { /* RST on already-forgotten stream */ }
    else if (!registered) {
      /* very early RST, drop */
      h2_stream_set_monitor(stream, NULL);
      h2_stream_rst(stream, H2_ERR_STREAM_CLOSED);
      h2_stream_dispatch(stream, H2_SEV_EOS_SENT);
      m_stream_cleanup(m, stream);
      m_be_annoyed(m);
    }
    /* ... */
}
```

`m_stream_cleanup` queues the stream twice for cleanup

```C
static void m_stream_cleanup(h2_mplx *m, h2_stream *stream)
{
    h2_conn_ctx_t *c2_ctx = h2_conn_ctx_get(stream->c2);

    /* ... unsubscribe, drop from registries ... */

    if (c2_ctx) {
        if (!stream_is_running(stream)) {
            APR_ARRAY_PUSH(m->spurge, h2_stream *) = stream;
        }
        else {
            h2_c2_abort(stream->c2, m->c1);
            h2_ihash_add(m->shold, stream);
        }
    }
    else {
        /* never started */
        APR_ARRAY_PUSH(m->spurge, h2_stream *) = stream;
    }
}
```

Later: draining the purge list frees each stream

```c
static void c1_purge_streams(h2_mplx *m)
{
    h2_stream *stream;
    int i;

    for (i = 0; i < m->spurge->nelts; ++i) {
        stream = APR_ARRAY_IDX(m->spurge, i, h2_stream*);
        ap_assert(stream->state == H2_SS_CLEANUP);
        /* ... */
        h2_stream_destroy(stream);  /* eventually calls free() */
    }
    apr_array_clear(m->spurge);
}
```

Summary

- Subtle logic error in stream cleanup allows the same stream to be cleaned up twice
- Logic is spread over many functions that seem right at surface level
- Modern vulnerabilities can be pretty complicated

Review: Heap corruption

- The heap is just a data structure like any other in the process address space
- In languages like C and C++, manual memory management can lead to this heap being corrupted
- Heap corruption bugs are everywhere, even in well-maintained, security-hardened systems

Additional resources

- sploitfun, Understanding glibc malloc (2015) — chunk layout and bins, walked through with real glibc source
- Phrack 57, Vudo malloc tricks and Once upon a free() — the original unlink-based heap exploitation write-ups
- shellphish, how2heap (GitHub) — runnable examples of every major glibc heap technique

For next time

- No reading
- Finish up assignment 2